



安徽大学
人工智能学院
School of Artificial Intelligence
Anhui University

《智能机器人系统综合训练》课程报告

学 院 人工智能

专 业 机器人工程

学生姓名学号 应宇杰 WA2224137

指导老师 常灯祥

课程编号 _____

课程学分 _____

提交日期 2025.12.30

项目仓库 <https://github.com/vyjjg/P-LINS.git>

目录

一、 实验目的和要求.....	3
二、 实验原理.....	4
三、 实验方法与内容.....	7
四、 实验原始纪录.....	8
五、 实验结果及分析.....	9
六、 实验总结与思考.....	12

一、 实验目的和要求

（一）实验目的

本实验旨在通过分析和调试基于扩展卡尔曼滤波（ESKF）的多传感器融合 SLAM 系统，使学生理解同步定位与地图构建的核心原理。具体包括：掌握 IMU、激光雷达传感器的数据特性及在 SLAM 中的互补作用；理解 ESKF 的状态估计流程（预测与更新）、噪声矩阵配置对滤波精度的影响；熟悉点云去畸变、平面特征提取等工程实现细节，建立从传感器数据到机器人位姿估计的完整认知。本实验报告中将侧重于介绍 lidar 配准方法的原理介绍，并且对整个工程部分实现的细节进行介绍，最后把整个工程在 WHU-helmet 数据集中的 street-bag 上跑通，给出部分建图的点云照片，对比使用 pdr 行人模型进行观测和单纯的 lio 算法的定位精度。

（二）实验要求

搭建 ROS 开发环境，编译运行 P-LINS 工程，正确解析激光雷达点云、IMU 数据的订阅与发布逻辑，验证各传感器数据在系统中的流转路径。测试 PDR（行人航位推算）速度观测、SE3 位姿观测等模块的融合效果，通过可视化工具（如 RViz）对比融合前后的

轨迹误差，评估多传感器融合策略的有效性。

二、 实验原理

（一）点云预处理

1. 原始数据筛选与去噪

剔除 LiDAR 原始数据中的无效点和噪声点，避免干扰后续处理，主要有两个操作：无效点主要是 lidar 设备的探测点的范围，可以剔除小于最小探测范围和大于最大探测范围的点。我们认为噪声点是相邻时间过短的两帧点云。

2. 运动畸变修正

由于对于一般 lidar 设备，无法保证同一帧收到的点云数据是在同一个时空坐标系下的，所以我们需要利用 IMU 的高频传感器数据计算得到的先验位姿，将每一帧中的点云转换到同一时空下，以消除点云的畸变，在本工程中是将同一帧的点云数据全部都转化到帧尾时空下，如式 1：

$$p_{corrected} = R_{finally}^T \cdot (R_i \cdot p_{raw} + t_i - t_{finally}) \quad 1$$

其中， p_{raw} 是第 i 时刻下的点云数据， (R_i, t_i) 表示第 i 时刻设备的位姿， $(R_{finally}, t_{finally})$ 表示帧尾时刻设备的位姿（相对于帧头 0 时刻）。

（二）特征提取方法

本工程先对去除畸变点云数据进行特征提取，先提取结构化信息较强的点云数据，再对特征点进行点到平面的 ICP 配准方法，特征提取的步骤按照计算点的局部曲率→剔除遮挡点和平行点→提取边缘点（角点）与平面点（面点）。

1. 计算点的局部曲率

在实际工程中计算点的局部曲率采用简化的方法，如式 2 所示：

$$\kappa(i) = \left[\left(\sum_{k=1}^5 r(i-k) + \sum_{k=1}^5 r(i+k) \right) - 10r(i) \right]^2 \quad 2$$

其中， $\kappa(i)$ 为第*i*个点的简化后的曲率， $r(i)$ 为第*i*个点的深度。简化后的曲率计算方法是通过计算某一个点的深度与前后五个点的深度的差异来衡量曲率大小的。

2. 剔除遮挡点和平行点

遮挡点剔除策略：先判断相邻两个点是否在同一条扫描线上，若在同一条扫描线上，那么就判断这两个点的深度距离差是否大于给定的阈值，若大于一定阈值，则将被遮挡的那个点及其后面五个点均认为被遮挡。**平行点剔除策略：**通过计算前后相邻点的测距值相对偏差率是否超过一定阈值，来判断是否为平行点。

3. 提取边缘点（角点）与平面点（面点）

将每根激光扫描线的点云划分为 6 个分段，保证特征点在空间中均匀分布，每个分段固定提取 20 个曲率最大的点，作为边缘点特征点，将非边缘点全部作为平面点，再通过 PCL 库的栅格法降采样对平面点进行降采样。

（三）K 近邻算法和 K_d 树

K 近邻算法核心思想是：对于一个待查询样本（目标点），在样本集中找到与它距离最近的 K 个样本（近邻点），这个算法在 lidar 的点云配准算法中有广泛的应用，常见的 K 近邻算法有暴

力 K 近邻、栅格法 K 近邻和 K-d 树法，本工程使用 K-d 树法。

K-d 树是一种用于组织 K 维空间数据的平衡二叉搜索树，其内部节点存储“切分维度”和“切分值”，用于划分空间，其叶子节点存储具体的点云数据，它通过旋转等操作保持二叉树平衡，确保最坏情况下的搜索效率仍为 $O(\log N)$ ，其核心流程分为**构建**和**搜索**。

1. K-d 树的构建

- (1) 选择切分维度：计算 x、y、z 三个维度的方差，选择方差最大的维度（数据分布最分散，切分效果最好）作为当前切分维度；
- (2) 确定切分值：在该切分维度上，选取数据的中值作为切分值，将点云分为左右两个子集（小于中值的为左子树，大于中值的为右子树）；
- (3) 递归构建：对左右两个子集，重复步骤 1-2，直到子集内点数量小于预设阈值（如 1 个点），停止递归，构建为叶子节点；
- (4) 生成平衡 K-d 树：通过调整节点位置，保证树的高度最小，实现平衡特性。

2. K-d 树的搜索

- (1) 向下遍历，找到最近叶子节点：从根节点出发，根据待查询点的各维度值与节点切分值的比较，依次向下遍历二叉树（如 x 轴切分，待查询点 $x <$ 切分值则走左子树，反之走右子树），最终找到包含待查询点的叶子节点，该节点对应的点作为初始近邻点；
- (2) 向上回溯，更新 K 近邻集合：回溯遍历过的节点，计算待查询点

与该节点对应点的距离，若该距离小于当前 K 近邻集合中的最大距离，则替换最大距离点，更新近邻集合；

- (3) 剪枝判断，减少无效搜索：对于回溯过程中的节点，计算待查询点到该节点切分空间的“最小距离”，若该最小距离大于当前 K 近邻集合的最大距离，则该节点对应的子树中不可能存在更近的点，直接剪枝（跳过该子树的搜索）；
- (4) 输出结果：当回溯至根节点时，停止搜索，最终的 K 近邻集合即为所求。

（四）点到平面 ICP 方法

ICP 算法的本质是通过迭代优化，求解两组点云之间的最优刚体变换（旋转矩阵 R + 平移向量 t ），使得两组点云的匹配误差最小。

常见的 ICP 配准方法有点到点、点到线和点到面的方法，本工程使用的是点到面的配准方法，它的核心思想是针对当前点云 P 中的每一个特征点 p_i ，在参考点云 Q 中找到 k 个近邻点，对这 k 个点通过最小二乘法拟合出一个局部平面，目标是 minimized 位姿变化后的点到这个平面的距离，将问题转化为一个优化问题，优化函数为：

$$T^* = \arg \min_{R,t} J(R,t) = \frac{1}{2} \sum_{i=1}^N f_i^2 = \frac{1}{2} \sum_{i=1}^N |n_i^T \cdot (R \cdot p_i + t - q_i)|^2 \quad 3$$

其中 T^* 为最终的优化函数， n_i^T 是每一个点的近邻拟合的平面的单位法向量的转置。由于这个优化函数是一个非线性函数，所以一般采用牛顿-高斯法对它进行迭代求解，这里不做赘述。

三、 实验方法与内容

本次实验将对 P-LINS 算法和不使用 PDR 模型作为观测的 LIO 算法在 WHU-helmet 数据集中的 street2 bag 数据上的定位结果，实验在 ubuntu 20.04 ros1 平台上进行。

四、 实验原始纪录

完整项目：<https://github.com/yyjgg/P-LINS.git>

这里展示 slam 算法主要流程调用函数结构源码：

```
void run()
{
    double t1 = omp_get_wtime();
    if (!ExtraFeature())
        return;

    // 更新当前匹配结果的初始位姿
    // 当前帧下采样
    DownsampleCurrentScan();
    // 对点云配准进行优化问题构建求解
    double t2 = omp_get_wtime();
    if (!Scan2MapOptimization())
        return;

    // std::cout << "[~~~~~ Scan2MapOptimization ]: time: " << omp_get_wtime() - t2
    << std::endl;

    lidar_pose = Trans2SE3d(transformTobeMapped);
    if (surf_score_T != surf_score_T){
        surf_score_T = 1000000;
    }

    if (surf_score_R != surf_score_R)
        surf_score_R = 1000000;
    if(!correct_t || !correct_R){

    }

    // eskf_tool->ObserveSE3(lidar_pose, 1 / 100000, 1 / 100000);
    eskf_tool->ObserveSE3(lidar_pose, surf_score_T / 1000, surf_score_R / 10000);
    cur_kf_pose = eskf_tool->GetNominalSE3();
    SE3d2Trans(cur_kf_pose, transformTobeMapped);
    // 根据配准结果确定是否是关键帧
    SaveKeyFramesAndFactor();
    // 将 lidar 里程计信息发送出去
```



```

PublishOdometry();

// 发送可视化点云信息
PublishFrames();

// std::cout<<"[~~~~~ ALL ]: time: "<< omp_get_wtime() - t1<<std::endl;

}

```

五、 实验结果及分析

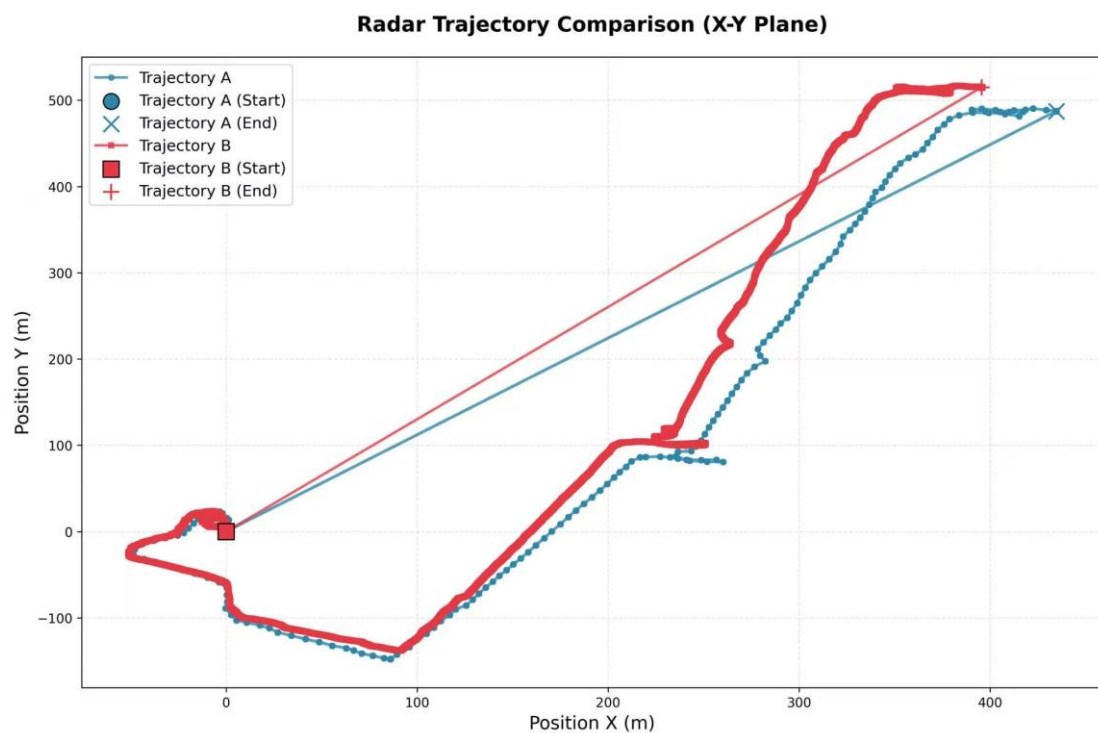


图 5.1 P-LINS 和 GNSS 的定位路径对比（2D）

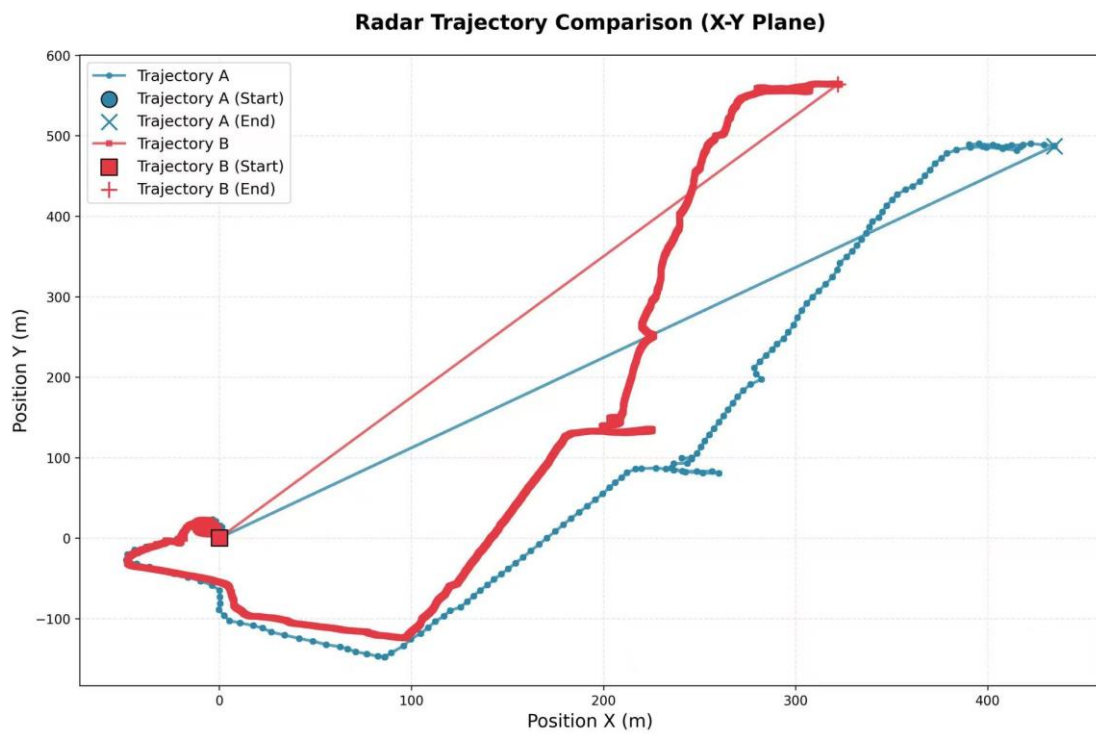


图 5.2 LIO 和 GNSS 的定位路径对比（2D）

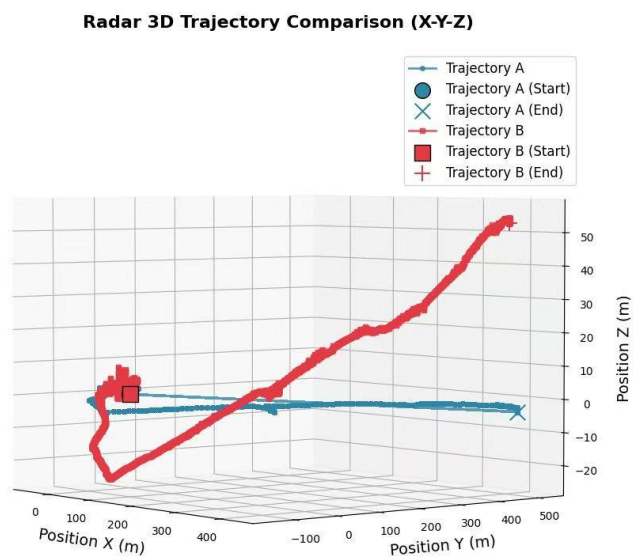


图 5.3 P-LINS 和 GNSS 的定位路径对比（3D）

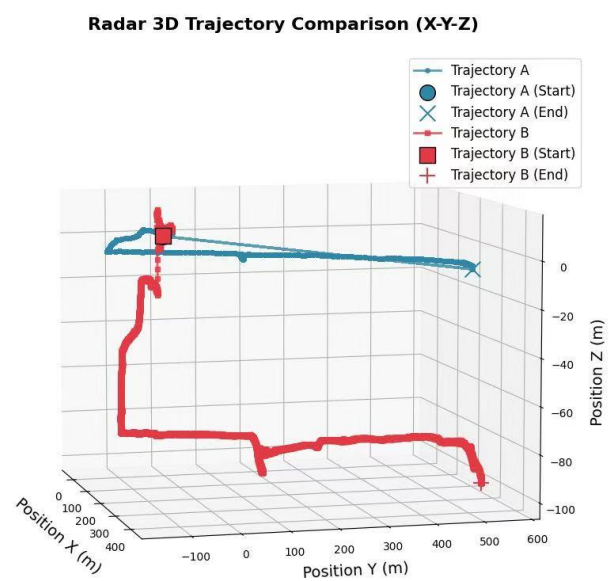


图 5.4 LIO 和 GNSS 的定位路径对比（3D）

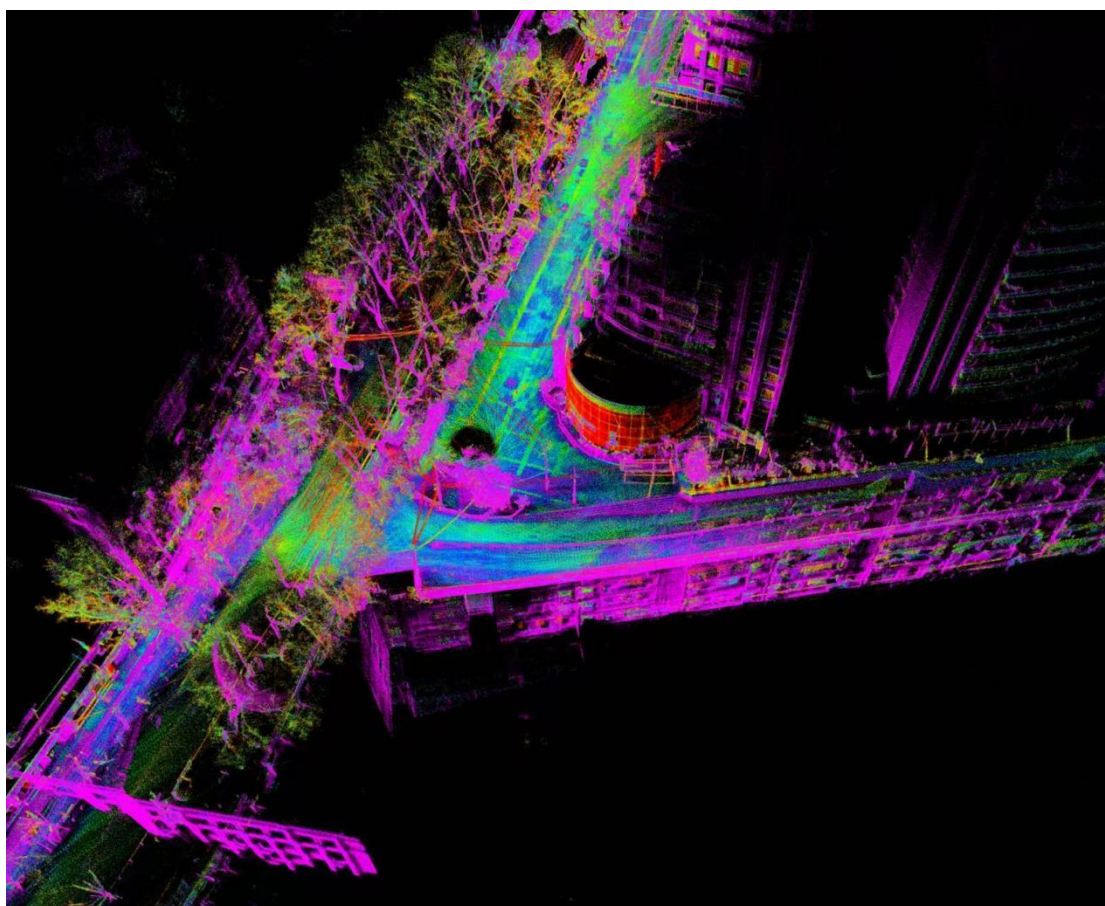


图 5.5 P-LINS 算法的局部点云地图

通过对比 P-LINS 和 LIO 算法，可以看出在 P-LINS 可以提高定位的精度，但是工程中仍存在问题，定位精度仍存在很大误差，还需要进一步调整参数和改进算法，下一步笔者准备针对 PDR 模型对象进行优化，还可以进一步增加回环检测，最终实现无轮速计辅助的 SOTA-LIO 算法。

六、 实验总结与思考

本次实验成功在 ROS 环境中编译运行 P-LINS 工程，验证了基于 ESKF 的多传感器融合 SLAM 系统的核心逻辑，深入掌握了点云预处理、特征提取、K-d 树近邻搜索及点到平面 ICP 配准的原理与工程实现。通过在 WHU-helmet 数据集上的测试对比，证实了 PDR 行人模型的融合能有效提升定位精度，相比单纯 LIO 算法，P-LINS 在轨迹一致性和建图效果上更具优势。实验也发现系统仍有优化空间，后续可通过细化 PDR 模型参数、增加回环检测模块，进一步提升无轮速计辅助场景下的定位鲁棒性与全局一致性。